

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Санкт-Петербург

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

по дисциплине «Методы обработки изображений»

Формирование изображения методом трассировки путей

Низкочастотная и билатеральная фильтрация шума синтезированного изображения

Выполнили: Рублёв Валерий Георгиевич, 409463

Казакова Кристина Дмитриевна, 470150

Преподаватель: Жданов Дмитрий Дмитриевич

Санкт-Петербург, 2026

Цель работы

Цель работы – реализовать синтез изображения трехмерной сцены методом трассировки путей, получить изображение с прямым и непрямым освещением, а затем снизить шум Монте-Карло с помощью низкочастотных и билатеральных фильтров без нарушения физического смысла яркостей.

Дополнительная цель текущей итерации – показать работу фильтрации на малом числе samples per pixel (SPP), где шум заметен визуально, и сравнить исходное и отфильтрованное изображения в интерфейсе программы.

Постановка задачи

В работе требуется:

- задать сцену треугольной геометрией и поддержать загрузку OBJ;
- реализовать трассировку путей для диффузных и зеркальных материалов;
- учитывать протяженные и точечные источники света;
- выполнять накопление изображения по SPP в GPU progressive режиме;
- сохранять результат в PPM, PNG и HDR;
- хранить фильтруемые значения в линейной яркости, а не в экранном sRGB;
- реализовать набор фильтров из темы низкочастотной фильтрации;
- отдельно реализовать билатеральную фильтрацию с сохранением границ объектов;
- обеспечить переключение фильтров в GUI и сравнение исходного изображения с отфильтрованным.

Общая схема решения

Реализация разделена на несколько частей.

- `models` – структуры сцены: камера, материалы, треугольники, источники, параметры рендера и фильтрации.
- `io` – загрузка JSON-конфигурации и импорт OBJ-геометрии.
- `services/path_tracer.py` – построение сцены и CPU/GPU через torch path tracing.
- `services/taichi_progressive_path_tracer.py` – progressive Monte-Carlo path tracing на Taichi + Metal.
- `services/image_filters.py` – отдельный сервис фильтрации: box, arithmetic, gaussian, median, bilateral.
- `ui/main_window.py` – интерфейс, накопление SPP, переключение фильтров и режим сравнения до/после.

Принципиально фильтры вынесены в отдельный сервис. Это соответствует архитектуре реального проекта: рендерер отвечает за физическое получение radiance, а постобработка отвечает только за преобразование уже полученного буфера.

Математическая модель трассировки путей

Уравнение рендеринга

Для точки поверхности x и направления наблюдения ω_o оценивается исходящая яркость:

$$L_{o(x, \omega_o)} = L_{e(x, \omega_o)} + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_{i(x, \omega_i)} \cos \theta_i d\omega_i$$

где:

- L_o – исходящая яркость;
- L_e – собственное излучение поверхности;

- f_r – BRDF материала;
- L_i – входящая яркость;
- $\cos \theta_i$ – косинус угла между нормалью поверхности и направлением прихода света.

В программе интеграл оценивается методом Монте-Карло. Для каждого пикселя выпускается несколько случайных лучей, результат усредняется:

$$\hat{L} = \frac{1}{N} \sum_{k=1}^N L_k$$

где N – число samples per pixel.

Материалы

Материал содержит две составляющие:

- диффузную, описываемую ламбертовой BRDF;
- зеркальную, описываемую направленным отражением.

Для ламбертового отражения используется:

$$f_r = \frac{\rho_d}{\pi}$$

где ρ_d – диффузный цвет материала.

В коде дополнительно проверяется закон сохранения энергии:

$$\rho_d + \rho_s \leq 1$$

для каждой RGB-компоненты. Это ограничивает нефизический рост яркости при многократных отражениях.

Выбор события отражения

На каждом попадании луча выбирается событие: диффузное или зеркальное. Вероятности выбираются по средней яркости соответствующих компонент:

$$p_d = \frac{|\rho_d|}{|\rho_d| + |\rho_s|}$$

$$p_s = \frac{|\rho_s|}{|\rho_d| + |\rho_s|}$$

Если выбрана диффузная компонента, новое направление выбирается по cosine-weighted hemisphere sampling. Если выбрана зеркальная компонента, направление вычисляется как отражение падающего луча относительно нормали:

$$\omega_r = \omega_i - 2(\omega_i \cdot n)n$$

После выбора события вклад делится на вероятность этого события. Благодаря этому оценка остается несмещенной.

Русская рулетка

После минимальной глубины пути включается русская рулетка. Вероятность продолжения пути выбирается по максимальной компоненте текущего throughput:

$$p_{\text{survive}} = \text{clamp}(\max(T_r, T_g, T_b), 0.1, 0.95)$$

Если путь выживает, throughput делится на p_{survive} . Это сохраняет математическое ожидание вклада, но уменьшает среднюю длину путей.

Источники света и световой поток

В сцене используются два типа источников:

- протяженные треугольные источники;
- точечные источники.

Для importance sampling источникам каждому источнику назначается вес, пропорциональный световому потоку. Вероятность выбрать источник:

$$p_i = \frac{\Phi_i}{\sum_j \Phi_j}$$

Сумма в знаменателе берется по всем источникам сцены: по всем протяженным треугольникам и всем точечным источникам.

Протяженный источник

Светящийся треугольник считается ламбертовым area light. Его поток в реализации оценивается как:

$$\Phi_{\text{area}} = \pi A |(L_e)|$$

где:

- A – площадь треугольника;
- $|(L_e)|$ – средняя RGB-яркость emission;
- π появляется из интегрирования ламбертового излучения по полусфере.

При вычислении прямого освещения вклад выбранной точки на источнике равен:

$$L_{\text{area}} = L_e f_r \frac{\cos \theta_x \cos \theta_y}{r^2 p_{\text{area}}}$$

где:

- $\cos \theta_x$ – угол между нормалью освещаемой поверхности и направлением к источнику;
- $\cos \theta_y$ – угол между нормалью источника и направлением к освещаемой точке;
- r – расстояние между точками;
- p_{area} – плотность вероятности выборки источника и точки на нем.

Множитель

$$G(x, y) = \frac{\cos \theta_x \cos \theta_y}{r^2}$$

является геометрическим членом. Он учитывает наклон принимающей поверхности, видимую площадь источника и уменьшение телесного угла с расстоянием.

Точечный источник

Для точечного источника intensity трактуется как radiant intensity – мощность на единицу телесного угла. Полный поток:

$$\Phi_{\text{point}} = 4\pi |(I)|$$

где $|(I)|$ – средняя RGB-интенсивность источника. Множитель 4π соответствует полному телесному углу сферы.

Вклад точечного источника:

$$L_{\text{point}} = I f_r \cos \frac{\theta_x}{r^2 p_{\text{point}}}$$

Здесь нет множителя $\cos \theta_y$, потому что точечный источник не имеет площади и ориентации в текущей модели.

Накопление SPP и получение экранного изображения

GPU progressive режим работает следующим образом.

1. Пользователь задает `samples_per_pixel` в JSON.
2. Кнопка GPU PathTrace старт начинает накопление.
3. Каждый кадр добавляет один новый Monte-Carlo sample per pixel.
4. Изображение справа обновляется после очередного SPP.
5. Когда накоплено заданное число SPP, процесс останавливается автоматически.

Внутренний буфер рендера – это не экранная картинка, а линейная RGB radiance:

$$R(x, y) = [R_r, R_g, R_b]$$

Значения могут быть больше 1 и хранят физический смысл яркости до отображения на экран.

После фильтрации применяется tone mapping:

$$C_{\text{norm}} = \text{clamp}\left(\frac{R}{s}, 0, 1\right)$$

Затем гамма-коррекция:

$$C_{\text{display}} = 255 C_{\text{norm}}^{\frac{1}{\gamma}}$$

Только после этого массив становится обычной RGB-картинкой диапазона 0–255.

Математические модели фильтров

Фильтрация применяется к radiance-буферу до tone mapping и gamma correction. Это важно: фильтр работает с линейной яркостью, а не с уже искаженной экранной картинкой.

После вычисления результата фильтра используется коэффициент смешивания strength:

$$R_{\text{mix}} = (1 - \alpha)R_{\text{original}} + \alpha R_{\text{filtered}}$$

где:

- R_{original} – исходный накопленный RGB radiance;
- R_{filtered} – результат выбранного фильтра;
- α – параметр Сила в GUI, безразмерное число от 0 до 1.

При $\alpha = 0$ фильтр не влияет на изображение. При $\alpha = 1$ берется полностью отфильтрованный результат.

none

Фильтр none не изменяет изображение:

$$R_{\text{filtered}}(x, y) = R_{\text{original}}(x, y)$$

Этот режим нужен для сравнения и проверки исходного шума.

box / arithmetic

В данной лабораторной работе box и arithmetic являются синонимами. Оба реализуют арифметический усредняющий фильтр из лекции:

$$R_{\text{filtered}}(x,y) = \frac{1}{MN} \sum_{i=-r}^r \sum_{j=-r}^r R(x+i, y+j)$$

где:

- r – радиус окна;
- $M = N = 2r + 1$;
- все коэффициенты маски одинаковы;
- сумма коэффициентов равна 1.

Для $\text{radius} = 1$ используется окно 3×3 , для $\text{radius} = 2 - 5 \times 5$.

Такой фильтр хорошо подавляет шум, но размывает границы, потому что смешивает все пиксели окна с одинаковым весом.

gaussian

Гауссов фильтр также является низкочастотным фильтром, но соседние пиксели получают разные веса:

$$G(i, j) = \exp\left(-\frac{i^2 + j^2}{2\sigma_s^2}\right)$$

После нормировки:

$$w(i, j) = \frac{G(i, j)}{\sum_{a=-r}^r \sum_{b=-r}^r G(a, b)}$$

Результат:

$$R_{\text{filtered}}(x,y) = \sum_{i=-r}^r \sum_{j=-r}^r w(i, j) R(x+i, y+j)$$

Ближние пиксели влияют сильнее, дальние – слабее. Поэтому гауссов фильтр выглядит мягче, чем box, но также может размывать границы.

median

Медианный фильтр является нелинейным ранговым фильтром. Для каждого пикселя берется окно S размера $(2r + 1) \times (2r + 1)$.

В лекции алгоритм описан как сортировка значений яркости в окне и выбор центрального элемента. В реализации для цветного изображения используется яркость RGB-пикселя:

$$Y = 0.2126R + 0.7152G + 0.0722B$$

Далее значения Y в окне ранжируются, выбирается медианная позиция:

$$k = \left\lfloor |S| \frac{1}{2} \right\rfloor$$

и в результат записывается не отдельно медиана по каналам, а весь RGB-пиксель, соответствующий медианной яркости:

$$R_{\text{filtered}}(x,y) = R(q_k), \quad q_k = \text{median-rank}(S)$$

Так сохраняется логика лекции: после сортировки выбирается центральный пиксель. При этом цвет не собирается искусственно из независимых медиан каналов.

Для сохранения границ объектов, если доступен индекс объекта первого пересечения, медиана берется только среди пикселей того же объекта. Пиксели другого объекта в окне заменяются центральным

значением. Это не меняет идею медианного фильтра, но предотвращает смешивание разных поверхностей.

После медианного фильтра выполняется нормировка суммарной энергии по объектам:

$$\sum_{p \in O} R_{\text{filtered}(p)} = \sum_{p \in O} R_{\text{original}(p)}$$

Это соответствует требованию физически корректной фильтрации: фильтр может перераспределять яркость локально, но не должен создавать или уничтожать свет без основания.

bilateral

Билатеральный фильтр является основным фильтром для финального результата. Он усредняет соседние пиксели, но вес зависит не только от расстояния в изображении, а также от сходства значений.

Базовая формула:

$$R_{\text{filtered}(p)} = \frac{1}{W_p} \sum_{q \in S} R(q) w_{s(p,q)} w_{r(p,q)}$$

где:

$$W_p = \sum_{q \in S} w_{s(p,q)} w_{r(p,q)}$$

В реализации используются четыре типа весов:

1. пространственный вес:

$$w_s = \exp\left(-\frac{|p - q|^2}{2\sigma_s^2}\right)$$

2. вес по яркости:

$$w_c = \frac{1}{1 + \left(\frac{Y_q - Y_p}{\sigma_c}\right)^2}$$

3. вес по глубине:

$$w_d = \exp\left(-\frac{(D_q - D_p)^2}{2\sigma_d^2}\right)$$

4. вес по нормали:

$$w_n = \exp\left(-\frac{(1 - N_q \cdot N_p)^2}{2\sigma_n^2}\right)$$

Также используется индекс объекта:

$$w_o = 1, \text{ если } \text{object}_q = \text{object}_p$$

$$w_o = 0, \text{ иначе}$$

Итоговый вес:

$$w(p, q) = w_s w_c w_d w_n w_o$$

Такой фильтр сглаживает шум внутри одной поверхности, но почти не смешивает разные объекты, разные глубины и разные нормали. Поэтому в сравнении до/после он заметно уменьшает шум, сохраняя крупные границы объектов.

Радиус и сила фильтра

Параметр Радиус определяет размер окна:

- `radius = 1` – окно 3×3 ;
- `radius = 2` – окно 5×5 ;
- `radius = 3` – окно 7×7 .

Чем больше радиус, тем сильнее сглаживание и выше вычислительная стоимость. Также растет риск размытия деталей.

Параметр Сила – это коэффициент α в формуле смешивания:

$$R_{\text{mix}} = (1 - \alpha)R_{\text{original}} + \alpha R_{\text{filtered}}$$

Он не имеет физических единиц. Это ручной коэффициент, позволяющий управлять балансом между шумоподавлением и сохранением деталей.

Распараллеливание фильтрации

Задача фильтрации хорошо распараллеливается, потому что результат каждого выходного пикселя зависит только от локального окна входного изображения и не зависит от результата соседних выходных пикселей.

В проекте это использовано для медианного фильтра:

- изображение делится на независимые горизонтальные полосы;
- каждая полоса обрабатывается отдельным `worker`-поток;
- внутри полосы окна строятся векторизованно через `numpy.sliding_window_view`;
- медианный элемент выбирается через `argpartition`, что быстрее полной сортировки.

Алгоритмический смысл медианного фильтра при этом не меняется: для каждого пикселя остается окно, ранжирование значений по яркости и выбор центрального элемента.

Для GPU такая задача также естественна: можно запускать отдельный поток на каждый пиксель. Гауссов фильтр дополнительно допускает сепарабельную реализацию: горизонтальный проход и вертикальный проход. Билатеральный фильтр распараллеливается по пикселям, но в общем случае не является сепарабельным, потому что веса зависят от яркости, глубины, нормали и индекса объекта.

Реализованные возможности программы

В итоговой версии реализовано:

- GPU progressive path tracing через Taichi + Metal;
- накопление до заданного `samples_per_pixel` из JSON;
- автоматическая остановка после достижения целевого SPP;
- хранение исходного radiance и прямой компоненты первого попадания;
- построение guide-buffers: глубина, нормаль, индекс объекта;
- фильтры `none`, `box`, `arithmetic`, `gaussian`, `median`, `bilateral`;
- мгновенное применение фильтра к уже накопленному кадру без повторного рендера;
- режим сравнения до/после в GUI;
- экспорт PPM, PNG, HDR;
- проверка материалов на сохранение энергии.

Демонстрационная сцена

В качестве демонстрационной сцены используется открытая постановка на расширенном полу. В центре сцены расположены несколько простых геометрических объектов: синяя коробка, низкая коричневая тумба, светлый подиум, фиолетовый высокий блок, наклонная зеркальная панель и две пирамиды, загруженные из OBJ. Объекты имеют разные материалы: матовые, смешанные диффузно-зеркальные и более отражающие. Благодаря этому на одном кадре можно наблюдать не только прямое освещение, но и цветные переотражения, контактные тени и отражения.

Освещение сцены состоит из восьми маленьких протяженных источников под потолком и трех точечных цветных источников внутри сцены. Протяженные источники дают мягкие тени, а точечные цветные источники усиливают локальные цветовые вклады. Расширенный пол нужен для того, чтобы при повороте камеры сцена не обрывалась и можно было рассматривать композицию с разных сторон.

Сцена выбрана так, чтобы одновременно проявлялись:

- мягкие тени от area-light источников;
- цветные переотражения;
- зеркальная компонента;
- разные глубины и нормали для проверки билатерального фильтра;
- заметный шум при малом SPP.

Основные параметры:

- разрешение: 500×500
- целевое SPP в конфигурации: 12
- максимальная глубина пути: 4
- gamma: 2.20
- режим нормировки: percentile
- число треугольников: 64
- протяженных источников: 8
- точечных источников: 3.

Результаты

Для демонстрации самой сцены использованы нефильтрованные рендеры при большем числе SPP. Это позволяет показать геометрию, материалы и освещение без влияния постобработки. Отдельно ниже приведены сравнения фильтров на одном и том же шумном кадре малого SPP.

Для сравнения фильтров используется единый режим:

- $SPP = 4$;
- $radius = 2$;
- режим До/после включен.

Слева в каждом сравнении показан исходный Monte-Carlo результат, справа – результат выбранного фильтра. Такой формат позволяет оценить не только факт уменьшения шума, но и побочные эффекты: размытие границ, пятнистость медианного фильтра или сохранение границ у билатерального фильтра.

Медианный фильтр после оптимизации стал применяться быстро и наглядно демонстрирует ранговую фильтрацию. Он хорошо показывает принцип выбора центрального элемента в окне, однако для шума Монте-Карло уступает билатеральному фильтру: может появляться пятнистость, потому что Monte-Carlo шум не является простым импульсным шумом типа «соль/перец».

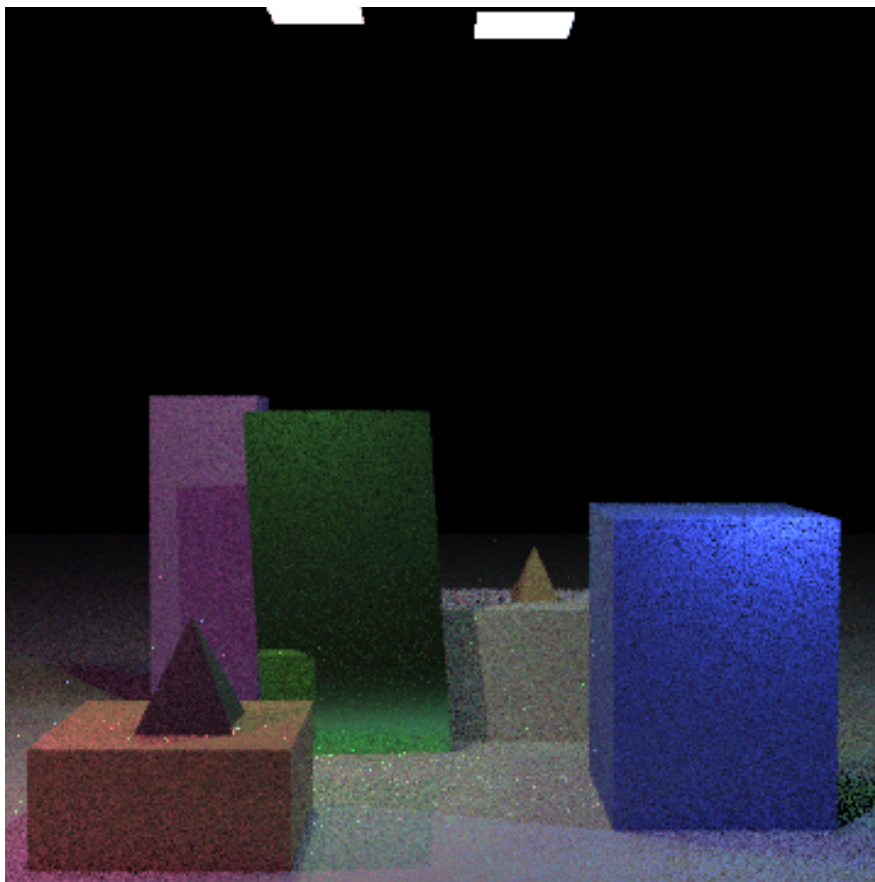


Рис. 1. Основной нефiltroванный ракурс сцены при 16 spp. Видны крупные объекты, цветные источники сверху, мягкие тени на полу и разноцветные переотражения.

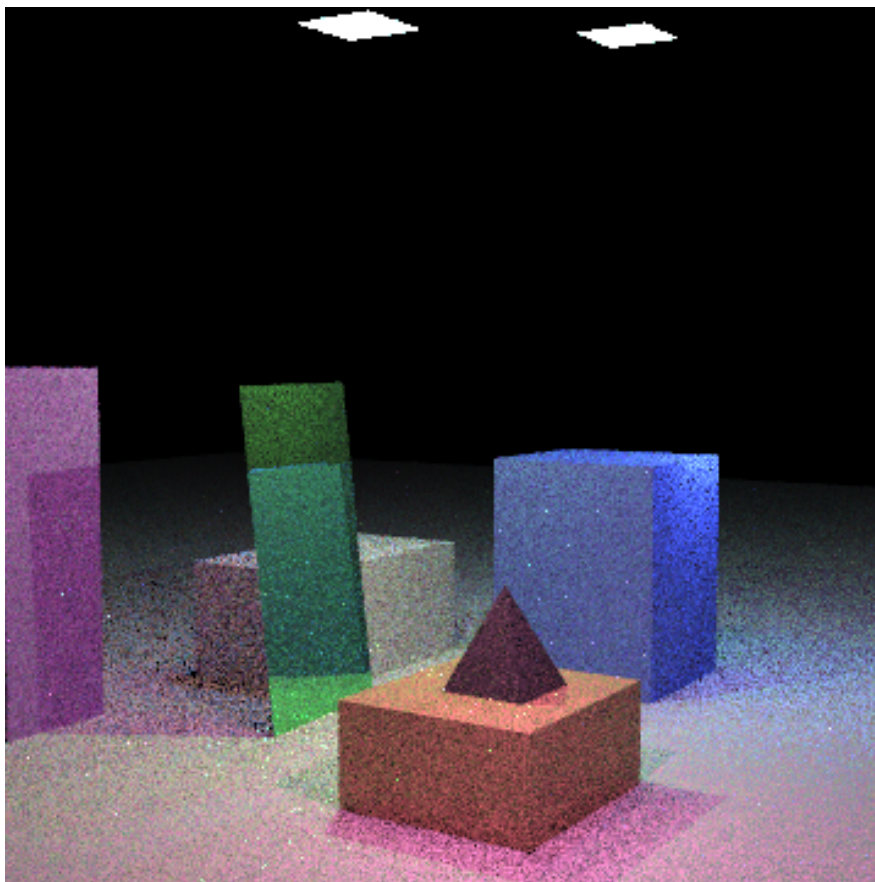


Рис. 2. Левый нефiltroванный ракурс при 16 spp. При смене положения камеры сохраняется та же геометрия и освещение, но лучше читается взаимное расположение тумбы, пирамиды, фиолетового блока и задних объектов.

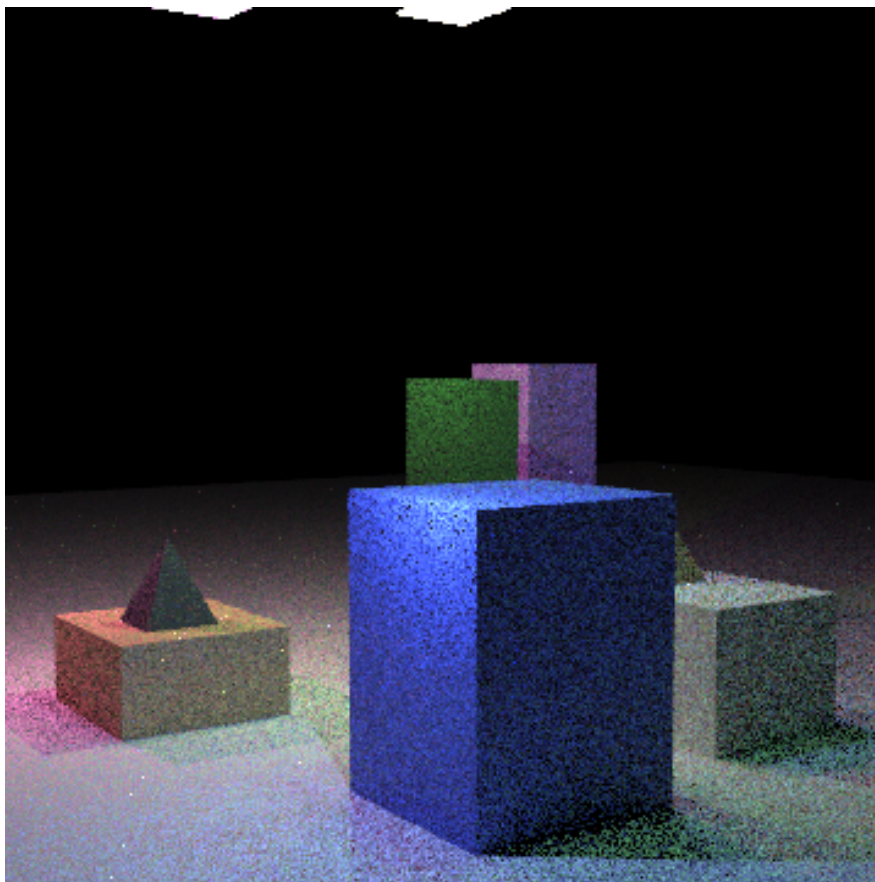


Рис. 3. Правый нефiltroванный ракурс при 16 spp. С этого положения заметнее объем синей коробки, светлого подиума и влияние цветного освещения на разные поверхности.

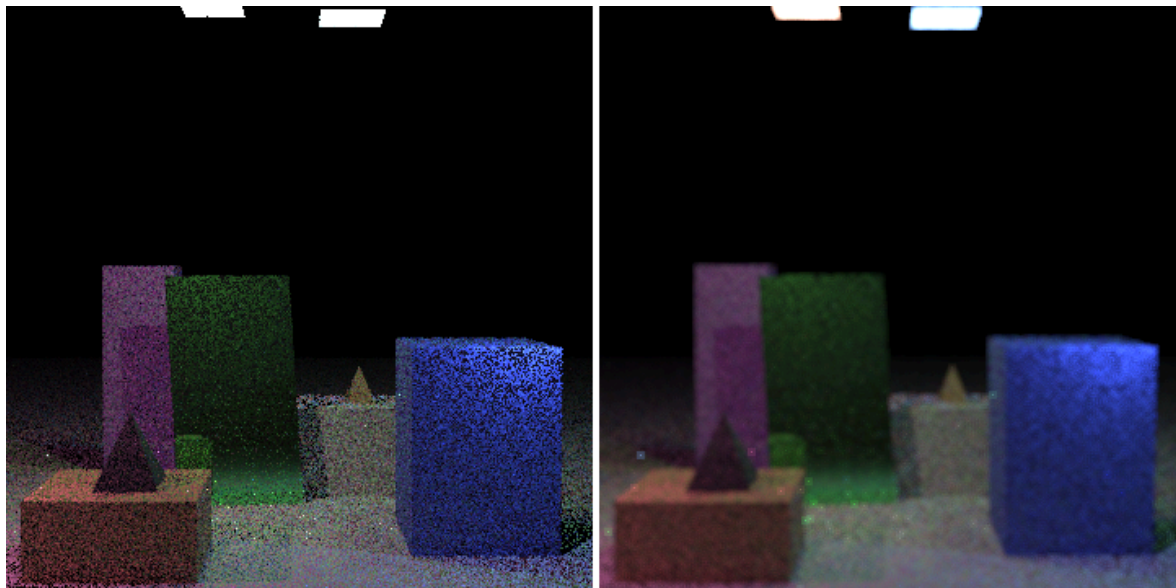


Рис. 4. Фильтр box, radius = 2. Простое равномерное усреднение заметно снижает шум, но вместе с ним размывает границы и мелкие детали.

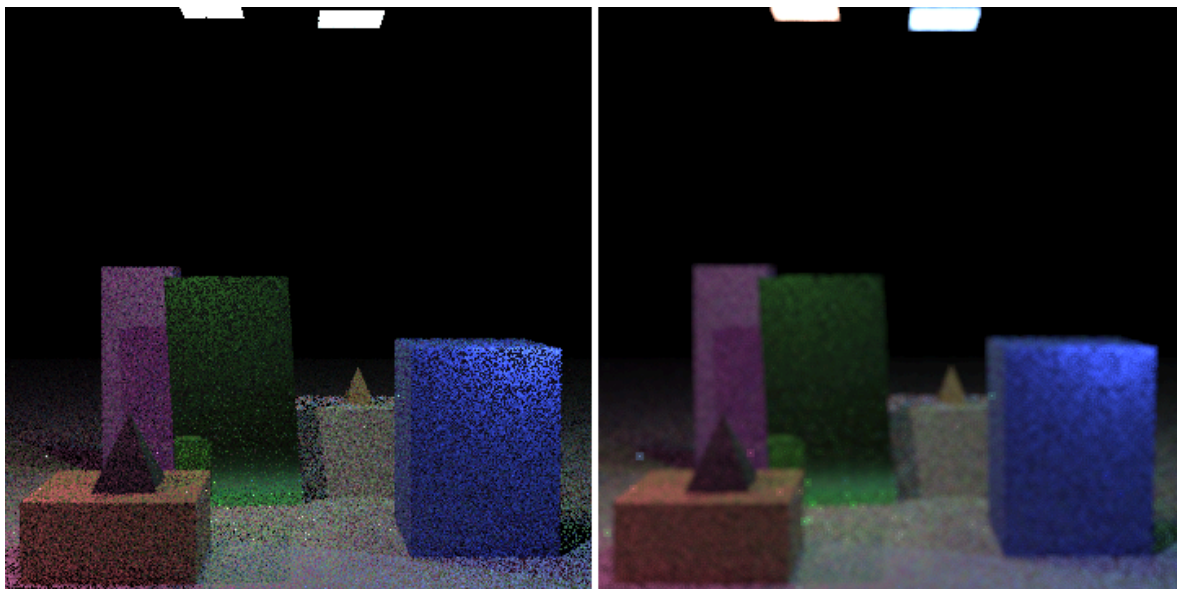


Рис. 5. Фильтр `arithmetic`, `radius = 2`. В реализации он является синонимом `box`: используется арифметическое среднее по окну с равными коэффициентами.

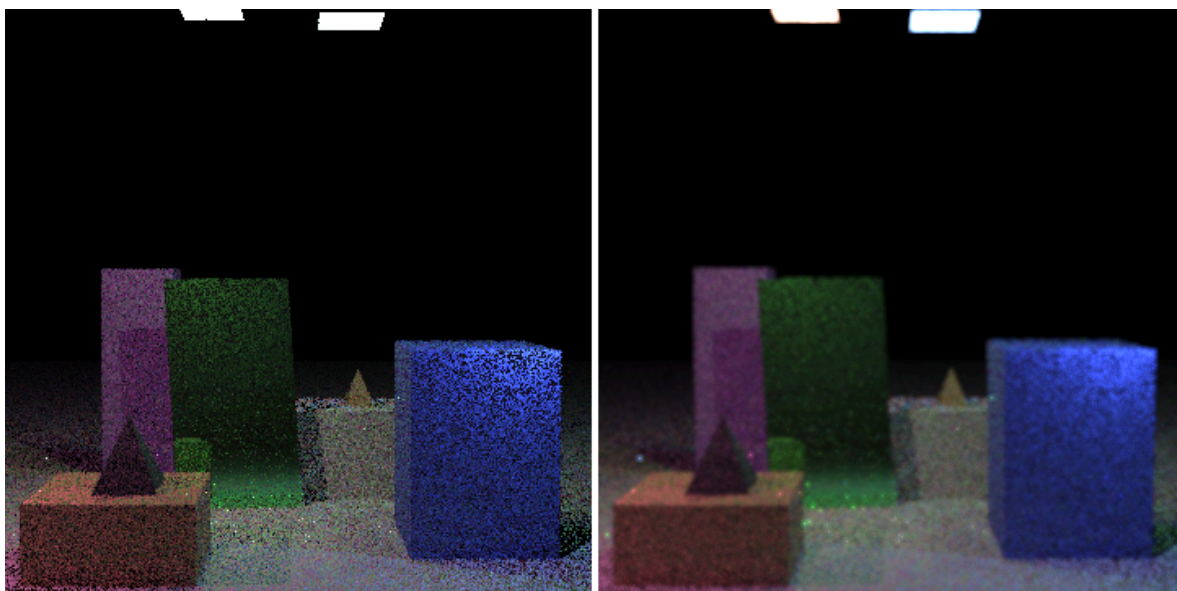


Рис. 6. Фильтр `gaussian`, `radius = 2`. Гауссово взвешивание сглаживает шум мягче, чем равномерное среднее, но границы объектов всё равно частично размываются.

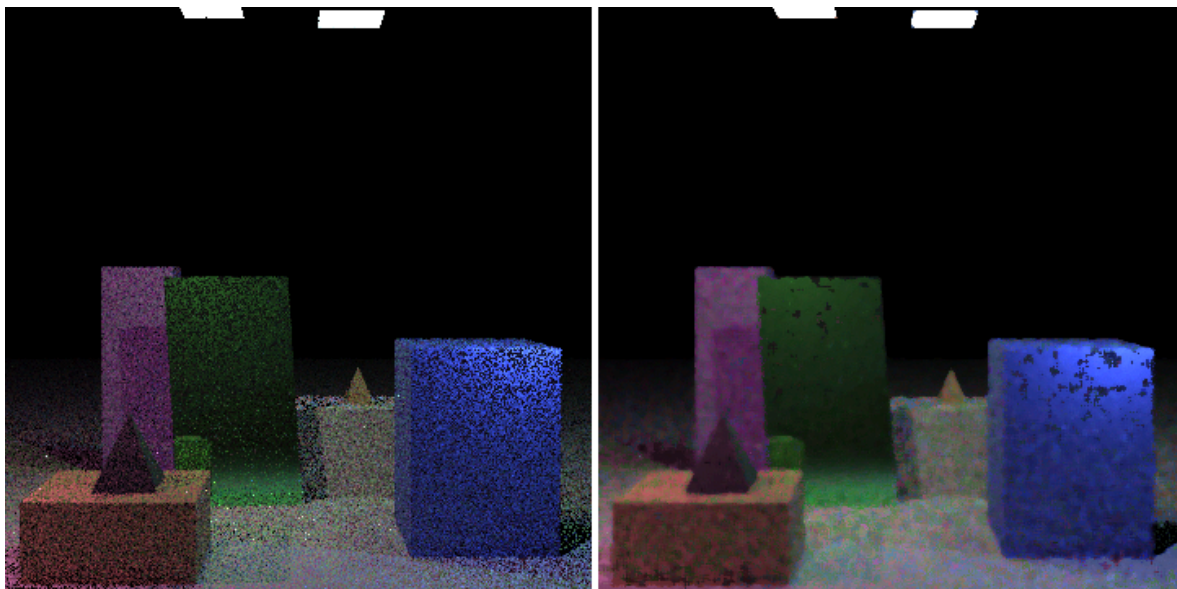


Рис. 7. Фильтр `median`, `radius = 2`. Ранговая фильтрация выбирает медианный пиксель в окне и хорошо демонстрирует нелинейную обработку, но для Monte-Carlo шума может давать пятнистость.

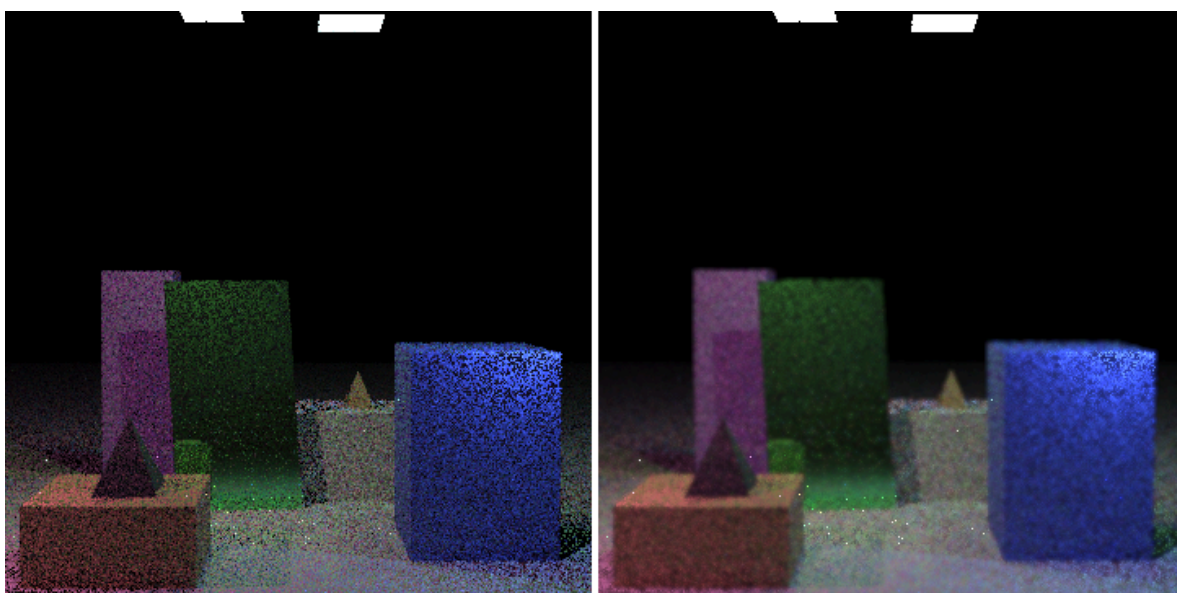


Рис. 8. Фильтр `bilateral`, `radius = 2`. Шум снижается внутри поверхностей, а границы объектов сохраняются лучше за счет учета яркости, глубины, нормали и индекса объекта.

Проверка корректности

Корректность проверялась на нескольких уровнях.

1. Материалы проверяются на сохранение энергии: сумма диффузной и зеркальной компонент не превышает единицы.
2. Для протяженных и точечных источников используются согласованные веса по световому потоку.
3. При прямом освещении `area light` учитывается геометрический множитель $\cos \theta_x \cos \frac{\theta_y}{r^2}$.
4. Фильтрация выполняется до `tone mapping`, то есть в линейной `radiance`.
5. Для медианного и билатерального фильтра сохраняется суммарная энергия по объектам.
6. Автоматические тесты проверяют геометрию, выбор вероятностей источников, сохранение энергии фильтров и корректность медианной фильтрации по объектам.

Вывод

В лабораторной работе реализован path tracer для треугольной сцены с глобальным освещением. Программа поддерживает диффузные и зеркальные материалы, OBJ-геометрию, протяженные и точечные источники, русскую рулетку, антиалиасинг и экспорт результатов в несколько форматов.

Основным расширением текущей версии стала физически осмысленная фильтрация шума. Фильтры работают с линейным radiance-буфером до tone mapping. Были реализованы простые низкочастотные фильтры, медианный фильтр и билатеральный фильтр с использованием глубины, нормали и индекса объекта. На малых SPP режим До/после показывает, что билатеральный фильтр уменьшает шум Монте-Карло заметнее и аккуратнее, чем обычное усреднение, так как сохраняет границы объектов.

Медианный фильтр был оптимизирован и распараллелен без изменения алгоритмического смысла из лекции: для каждого пикселя сохраняется локальное окно, ранжирование по яркости и выбор центрального элемента. Это сделало фильтр пригодным для интерактивной демонстрации в GUI.

Таким образом, итоговая программа демонстрирует полный цикл: физически мотивированный синтез изображения, накопление SPP на GPU, хранение абсолютной яркости, интерактивное шумоподавление и сравнение исходного и отфильтрованного результата.